



**YENEPOYA**  
Deemed to be University

**YENEPOYA**  
**(DEEMED TO BE UNIVERSITY)**

**Project – Low Level Design**

**on**

**Edge AI for Manufacturing Real-time  
Monitoring**

**Student Name: MUHAMMED SUBAIR CH**

**Industry Mentor: Mr. Sumit K Shukla**

**Guided by: Mr Praveen**

## **TABLE OF CONTENT**

|                                       |    |
|---------------------------------------|----|
| 1. Introduction .....                 | 6  |
| 1.1. Scope of the document. ....      | 6  |
| 1.2. Intended Audience .....          | 6  |
| 1.3. System overview. ....            | 6  |
| 2. System Design. ....                | 6  |
| 2.1. Application Design .....         | 6  |
| 2.2. Process Flow. ....               | 6  |
| 2.3. Information Flow. ....           | 7  |
| 2.4. Components Design .....          | 8  |
| 2.5. Key Design Considerations .....  | 8  |
| 2.6. API Catalogue. ....              | 8  |
| 3. Data Design. ....                  | 8  |
| 3.1. Data Model .....                 | 8  |
| 3.2. Data Access Mechanism .....      |    |
| 3.3. Data Retention Policies.....     | 9  |
| 3.4. Data Migration .....             | 9  |
| 4. Interfaces .....                   | 9  |
| 5. State and Session Management ..... | 10 |
| 6. Caching .....                      | 10 |
| 7. Non-Functional Requirements .....  | 10 |
| 7.1. Security Aspects .....           | 10 |
| 7.2. Performance Aspects .....        | 10 |
| 8. References .....                   | 11 |

# 1. INTRODUCTION

The Low-Level Design (LLD) document provides a comprehensive technical blueprint of the AI-Based Edge Object Detection System. It describes the internal structure, processing logic, and interaction between various system components at an implementation level.

This system leverages Edge AI principles by executing all computations locally using a TensorFlow Lite model. It eliminates dependency on cloud infrastructure, ensuring minimal latency and enhanced data privacy, which is particularly important in healthcare environments.

The system is designed to capture real-time video input, process image frames, perform object classification, and store results efficiently.

## 1.1 Scope of the Document

This document includes:

- Detailed module-level design
- Execution workflow and control flow
- Algorithmic representation of processing steps
- Data structures and storage mechanisms
- Error handling strategies
- Performance optimization techniques

It serves as a reference for developers and evaluators to understand the system at a granular level.

## 1.2 Intended Audience

- Software developers implementing the system
- Technical reviewers and architects
- Academic evaluators and instructors
- Students and researchers

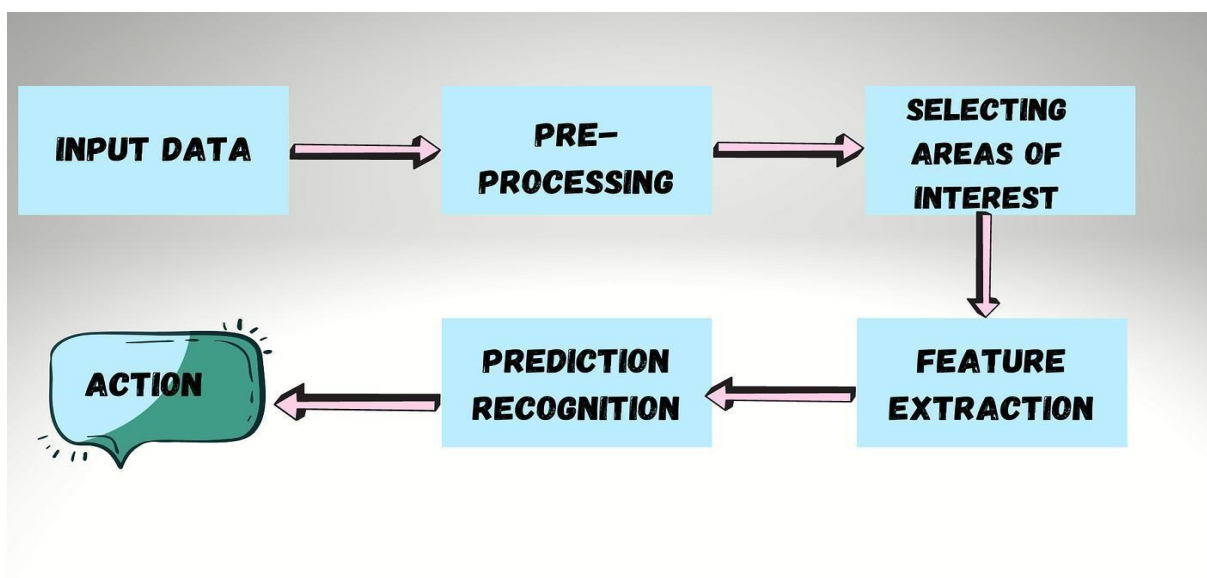
### 1.3 System Overview

The system operates as a real-time object classification pipeline using a pre-trained MobileNet TFLite model. It captures video frames via a webcam, processes them, and predicts object labels with associated confidence scores.

The results are displayed dynamically and recorded in a structured CSV format for further analysis.

## 2. SYSTEM DESIGN

### 2.1 Application Design

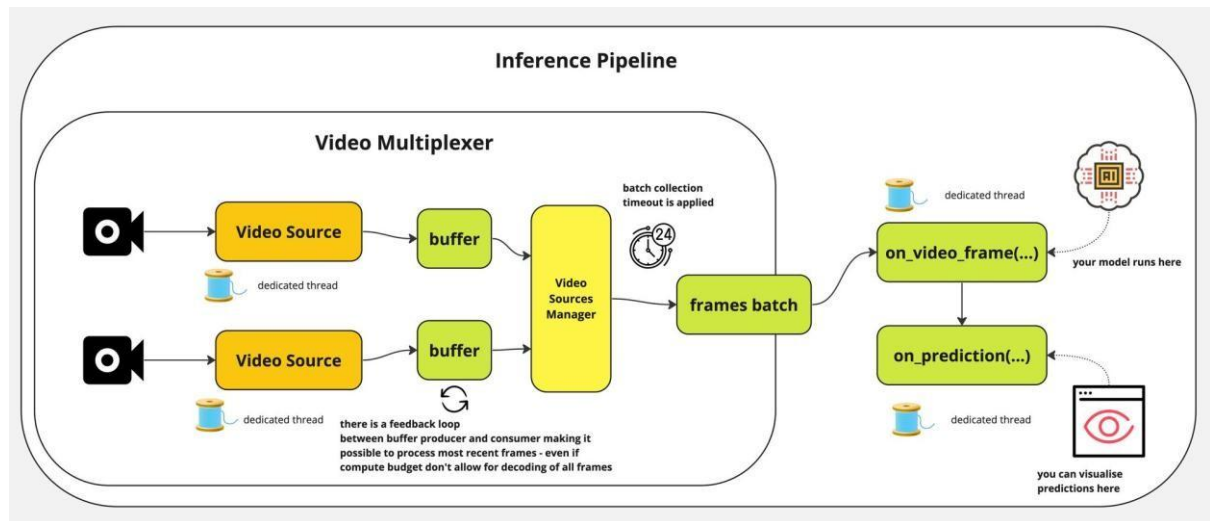


The application is structured using a modular layered architecture to ensure maintainability and scalability.

#### Layers:

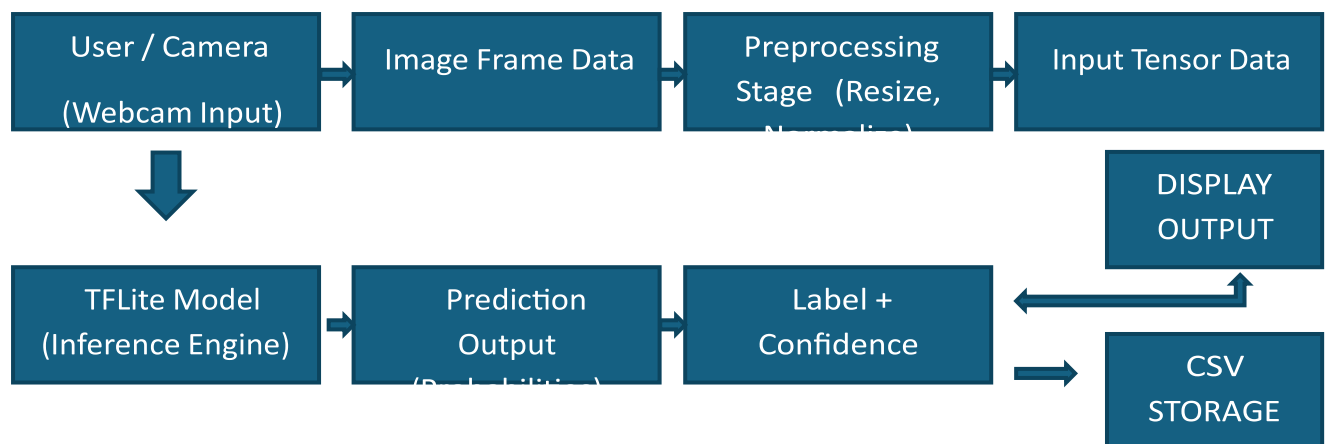
- **Input Layer:** Handles video capture using OpenCV
- **Preprocessing Layer:** Transforms raw input into model-compatible format
- **Inference Layer:** Executes TensorFlow Lite model
- **Output Layer:** Displays predictions and stores results

## 2.2 Process Flow



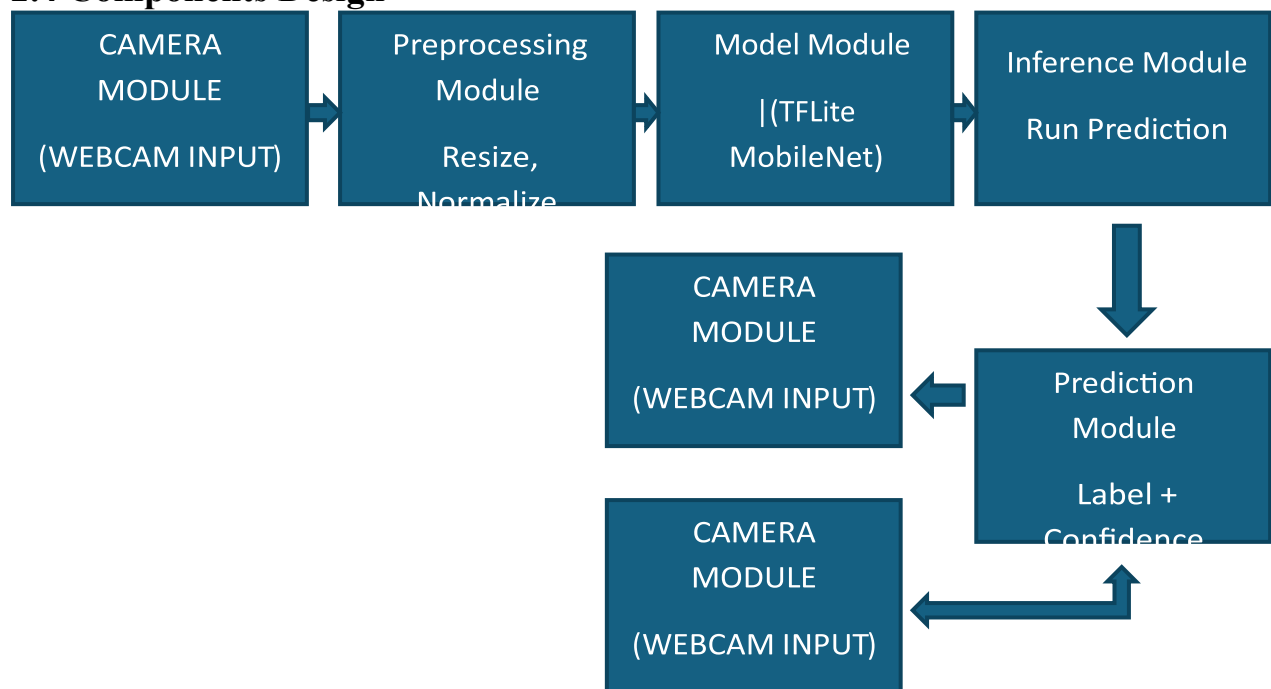
- System initialization and model loading
- Allocation of input and output tensors
- Continuous frame capture from webcam
- Image preprocessing (resize, normalize, reshape)
- Feeding input tensor into model
- Execution of inference
- Extraction of prediction results
- Filtering based on confidence threshold
- Rendering output on display
- Logging detection data into CSV file
- Loop execution until termination

## 2.3 Information Flow



- Input data is captured from the webcam
- Data is transformed into a structured tensor
- Tensor is processed by the inference engine
- Output probabilities are converted into labels
- Final results are displayed and stored

## 2.4 Components Design



### 2.4.1 Camera Module

Handles real-time frame acquisition. Ensures consistent frame rate and validates camera input.

### 2.4.2 Preprocessing Module

Performs:

- Image resizing (224×224)
- Color space conversion
- Normalization
- Tensor shaping

### 2.4.3 Model Module Responsible

for:

- Loading `.tflite` model
- Allocating tensors
- Managing interpreter lifecycle

### 2.4.4 Inference Module

Executes model prediction and ensures efficient CPU utilization.

### 2.4.5 Prediction Module Processes

output tensor:

- Identifies highest probability class
- Maps index to label
- Computes confidence score

### 2.4.6 Storage Module

Implements persistent logging:

- Writes structured records
- Maintains CSV format
- Ensures data consistency

### 2.4.7 Display Module Renders output:

- Displays label and confidence
- Updates in real-time
- Provides user feedback

## 2.5 Key Design Considerations

- Selection of lightweight model for edge devices
- Ensuring real-time inference capability
- Maintaining prediction stability

- Minimizing resource utilization
- Ensuring modularity and extensibility

## 2.6 API Catalogue

| API              | Input   | Output  | Description    |
|------------------|---------|---------|----------------|
| capture_frame    | None    | Frame   | Captures image |
| preprocess_frame | Frame   | Frame   | Resizes image  |
| detect_objects   | Frame   | Results | Runs YOLO      |
| extract_data     | Results | List    | Extract labels |
| display_output   | Frame   | None    | Shows UI       |
| save_data        | List    | CSV     | Stores logs    |

## 3. DATA DESIGN

### 3.1 Data Model

```
{
  "timestamp": "string",
  "object": "string",
  "confidence": "float"
}
```

### 3.2 Data Access Mechanism

The system uses file-based storage with CSV format. Data is appended in real-time and accessed through Python file handling mechanisms.

### 3.3 Data Retention Policies

- Data is stored locally
- No sensitive patient data is collected
- Files can be archived or deleted manually

### 3.4 Data Migration

Data can be exported and integrated into databases or analytics platforms for further processing.

## 4. INTERFACES

- **User Interface:** OpenCV display window
- **System Interface:** Communication between modules
- **External Interface:** Not applicable

## 5. STATE AND SESSION MANAGEMENT

The system is stateless in nature. Each frame is processed independently without dependency on previous frames.

## 6. CACHING

Currently, caching is not implemented. However, caching mechanisms can be introduced to optimize repeated predictions.

## 7. NON-FUNCTIONAL REQUIREMENTS

### 7.1 Security Aspects

- Complete local processing
- No external data transmission
- Ensures confidentiality

### 7.2 Performance Aspects

- Real-time processing capability
- Low latency inference
- Efficient CPU utilization
- Scalable for low-power devices

## 8. REFERENCES

- TensorFlow Lite Documentation
- OpenCV Documentation
- Python Official Documentation
- Edge AI Research Papers